Generalized Language Models

This PDF conversion (v1) was generated on October 10, 2025*

Online version: https://lilianweng.github.io/posts/2019-01-31-lm/
Date: **Estimated Reading Time:** min **Author:** Lilian Weng

[0]This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is https://lilianweng.github.io/posts/2019-01-31-lm/.

Contents

1 Generalized Language Models **3**

2 CoVe# **5**
2.1 NMT Recap# 5
2.2 Use CoVe in Downstream Tasks# 6

3 ELMo# **7**
3.1 Bidirectional Language Model# 7
3.2 ELMo Representations# 8
3.3 Use ELMo in Downstream Tasks# 8

4 Cross-View Training# **8**
4.1 Model Architecture# 9
4.2 Multi-Task Learning# 9
4.3 Use CVT in Downstream Tasks# 9

5 ULMFiT# **10**

6 GPT# **11**
6.1 Transformer Decoder as Language Model# 12
6.2 Byte Pair Encoding# 12
6.3 Supervised Fine-Tuning# 12

7 BERT# **15**
7.1 Pre-training Tasks# 16
7.2 Input Embedding# 20
7.3 Use BERT in Downstream Tasks# 21

8 ALBERT# **23**
8.1 Factorized Embedding Parameterization# 23
8.2 Cross-layer Parameter Sharing# 24
8.3 Sentence-Order Prediction (SOP)# 24

9 GPT-2# **24**
9.1 Zero-Shot Transfer# 24
9.2 BPE on Byte Sequences# 25
9.3 Model Modifications# 25

10 RoBERTa# **25**

11 T5# **26**

12 GPT-3# **28**

13 XLNet# **33**

14 BART# **37**

15 ELECTRA# **41**

16 Summary# **44**

17 Metric: Perplexity# **44**

18 Common Tasks and Datasets# **45**

19 Reference# **48**

20 Citation **50**

21 References **52**

Lil'Log

- |
- Posts
- Archive
- Search
- Tags
- FAQ

1 Generalized Language Models

Date: January 31, 2019 | Estimated Reading Time: 36 min | Author: Lilian Weng

Table of Contents

- CoVe
  - NMT Recap
  - Use CoVe in Downstream Tasks
- ELMo
  - Bidirectional Language Model
  - ELMo Representations
  - Use ELMo in Downstream Tasks
- Cross-View Training
  - Model Architecture
  - Multi-Task Learning
  - Use CVT in Downstream Tasks
- ULMFiT
- GPT
  - Transformer Decoder as Language Model
  - Byte Pair Encoding
  - Supervised Fine-Tuning
- BERT
  - Pre-training Tasks
  - Input Embedding

  - Use BERT in Downstream Tasks
- ALBERT
  - Factorized Embedding Parameterization
  - Cross-layer Parameter Sharing
  - Sentence-Order Prediction (SOP)
- GPT-2
  - Zero-Shot Transfer
  - BPE on Byte Sequences
  - Model Modifications
- RoBERTa
- T5
- GPT-3
- XLNet
- BART
- ELECTRA
- Summary
- Metric: Perplexity
- Common Tasks and Datasets
- Reference

[Updated on 2019-02-14: add ULMFiT and GPT-2.]
[Updated on 2020-02-29: add ALBERT.]
[Updated on 2020-10-25: add RoBERTa.]
[Updated on 2020-12-13: add T5.]
[Updated on 2020-12-30: add GPT-3.]
[Updated on 2021-11-13: add XLNet, BART and ELECTRA; Also updated the Summary section.]

We have seen amazing progress in NLP in 2018. Large-scale pre-trained language modes like OpenAI GPT and BERT have achieved great performance on a variety of language tasks using generic model architectures. The idea is similar to how ImageNet classification pre-training helps many vision tasks (*). Even better than vision classification pre-training, this simple and powerful approach in NLP does not require labeled data for pre-training, allowing us to experiment with increased training scale, up to our very limit.

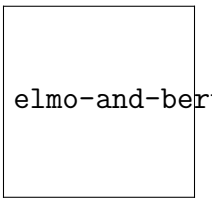


Figure 1: I guess they are Elmo & Bert? (Image source: here)

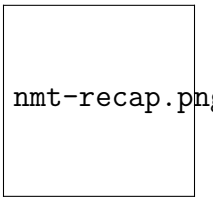


Figure 2: The NMT base model used in CoVe.

(*) He et al. (2018) found that pre-training might not be necessary for image segmentation task.

In my previous NLP post on word embedding, the introduced embeddings are not context-specific — they are learned based on word concurrency but not sequential context. So in two sentences, "*I am eating an apple*" and "*I have an Apple phone*", two "apple" words refer to very different things but they would still share the same word embedding vector.

Despite this, early adoption of word embeddings in problem-solving is to use them as additional features for an existing task-specific model and in a way the improvement is bounded.

In this post, we will discuss how various approaches were proposed to make embeddings dependent on context, and to make them easier and cheaper to be applied to downstream tasks in general form.

2 CoVe#

CoVe (McCann et al. 2017), short for **Contextual Word Vectors**, is a type of word embeddings learned by an encoder in an attentional seq-to-seq machine translation model. Different from traditional word embeddings introduced here, CoVe word representations are functions of the entire input sentence.

2.1 NMT Recap#

Here the Neural Machine Translation (NMT) model is composed of a standard, two-layer, bidirectional LSTM encoder and an attentional two-layer unidirectional LSTM decoder. It is pre-trained on the English-German translation task. The encoder learns and optimizes the embedding vectors of English words in order to translate them to German. With the intuition that the encoder should capture high-level semantic and syntactic meanings before transforming words into another language, the encoder output is used to provide contextualized word embeddings for various downstream language tasks.

Figure 3: The CoVe embeddings are generated by an encoder trained for machine translation task. The encoder can be plugged into any downstream task-specific model. (Image source: original paper)

- A sequence of n words in source language (English): $x = [x_1, \dots, x_n]$.
- A sequence of m words in target language (German): $y = [y_1, \dots, y_m]$.
- The GloVe vectors of source words: $\text{GloVe}(x)$.
- Randomly initialized embedding vectors of target words: $z = [z_1, \dots, z_m]$.
- The biLSTM encoder outputs a sequence of hidden states: $h = [h_1, \dots, h_n] = \text{biLSTM}(\text{GloVe}(x))$ and $h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$ where the forward LSTM computes $\overrightarrow{h}_t = \text{LSTM}(x_t, \overrightarrow{h}_{t-1})$ and the backward computation gives us $\overleftarrow{h}_t = \text{LSTM}(x_t, \overleftarrow{h}_{t-1})$.
- The attentional decoder outputs a distribution over words: $p(y_t \mid H, y_1, \dots, y_{t-1})$ where H is a stack of hidden states $\{h\}$ along the time dimension:

$$ \begin{aligned} \text{decoder hidden state: } s_t &= \text{LSTM}([z_{t-1}; \tilde{h}_{t-1}], s_{t-1}) \\ \text{attention weights: } \alpha_t &= \text{softmax}(H(W_1 s_t + b_1)) \\ \text{context-adjusted hidden state: } \tilde{h}_t &= \tanh(W_2[H^\top\alpha_t;s_t] + b_2) \\ \text{decoder output: } p(y_t\mid H, y_1, \dots, y_{t-1}) &= \text{softmax}(W_\text{out} \tilde{h}_t + b_\text{out}) \end{aligned} $$

2.2 Use CoVe in Downstream Tasks#

The hidden states of NMT encoder are defined as **context vectors** for other language tasks:

$$ \text{CoVe}(x) = \text{biLSTM}(\text{GloVe}(x)) $$

The paper proposed to use the concatenation of GloVe and CoVe for question-answering and classification tasks. GloVe learns from the ratios of global word co-occurrences, so it has no sentence context, while CoVe is generated by processing text sequences is able to capture the contextual information.

$$ v = [\text{GloVe}(x); \text{CoVe}(x)] $$

Given a downstream task, we first generate the concatenation of GloVe + CoVe vectors of input words and then feed them into the task-specific models as additional features.

Summary: The limitation of CoVe is obvious: (1) pre-training is bounded by available datasets on the supervised translation task; (2) the contribution of CoVe to the final performance is constrained by the task-specific model architecture.

Figure 4: The biLSTM base model of ELMo. (Image source: recreated based on the figure in ["Neural Networks, Types, and Functional Programming"](http://colah.github.io/posts/2015-09-NN-Types-FP/) by Christopher Olah.)

In the following sections, we will see that ELMo overcomes issue (1) by unsupervised pre-training and OpenAI GPT & BERT further overcome both problems by unsupervised pre-training + using generative model architecture for different downstream tasks.

3 ELMo#

ELMo, short for **Embeddings from Language Model** (Peters, et al, 2018) learns contextualized word representation by pre-training a language model in an *unsupervised* way.

3.1 Bidirectional Language Model#

The bidirectional Language Model (**biLM**) is the foundation for ELMo. While the input is a sequence of n tokens, $(x_1, \dots, x_n)$, the language model learns to predict the probability of next token given the history.

In the forward pass, the history contains words before the target token,

$$ p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid x_1, \dots, x_{i-1}) $$

In the backward pass, the history contains words after the target token,

$$ p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid x_{i+1}, \dots, x_n) $$

The predictions in both directions are modeled by multi-layer LSTMs with hidden states $\overrightarrow{\mathbf{h}}_{i,\ell}$ and $\overleftarrow{\mathbf{h}}_{i,\ell}$ for input token x_i at the layer level $\ell=1,\dots,L$. The final layer's hidden state $\mathbf{h}_{i,L} = [\overrightarrow{\mathbf{h}}_{i,L}; \overleftarrow{\mathbf{h}}_{i,L}]$ is used to output the probabilities over tokens after softmax normalization. They share the embedding layer and the softmax layer, parameterized by Θ_e and Θ_s respectively.

The model is trained to minimize the negative log likelihood (= maximize the log likelihood for true words) in both directions:

$$ \begin{aligned} \mathcal{L} = - \sum_{i=1}^n \Big(\log p(x_i \mid x_1, \dots, x_{i-1}; \Theta_e, \overrightarrow{\Theta}_\text{LSTM}, \Theta_s) + \\ \log p(x_i \mid x_{i+1}, \dots, x_n; \Theta_e, \overleftarrow{\Theta}_\text{LSTM}, \Theta_s) \Big) \end{aligned} $$

3.2 ELMo Representations#

On top of a L-layer biLM, ELMo stacks all the hidden states across layers together by learning a task-specific linear combination. The hidden state representation for the token x_i contains $2L+1$ vectors:

$$ R_i = \{ \mathbf{h}_{i,\ell} \mid \ell = 0, \dots, L \} $$

where $\mathbf{h}_{0, \ell}$ is the embedding layer output and $\mathbf{h}_{i, \ell} = [\overrightarrow{\mathbf{h}}_{i,\ell}; \overleftarrow{\mathbf{h}}_{i,\ell}]$.

The weights, $\mathbf{s}^\text{task}$, in the linear combination are learned for each end task and normalized by softmax. The scaling factor γ^task is used to correct the misalignment between the distribution of biLM hidden states and the distribution of task specific representations.

$$ v_i = f(R_i; \Theta^\text{task}) = \gamma^\text{task} \sum_{\ell=0}^L s^\text{task}_i \mathbf{h}_{i,\ell} $$

To evaluate what kind of information is captured by hidden states across different layers, ELMo is applied on semantic-intensive and syntax-intensive tasks respectively using representations in different layers of biLM:

- **Semantic task**: The *word sense disambiguation (WSD)* task emphasizes the meaning of a word given a context. The biLM top layer is better at this task than the first layer.
- **Syntax task**: The *part-of-speech (POS) tagging* task aims to infer the grammatical role of a word in one sentence. A higher accuracy can be achieved by using the biLM first layer than the top layer.

The comparison study indicates that syntactic information is better represented at lower layers while semantic information is captured by higher layers. Because different layers tend to carry different type of information, *stacking them together helps*.

3.3 Use ELMo in Downstream Tasks#

Similar to how CoVe can help different downstream tasks, ELMo embedding vectors are included in the input or lower levels of task-specific models. Moreover, for some tasks (i.e., SNLI and SQuAD, but not SRL), adding them into the output level helps too.

The improvements brought up by ELMo are largest for tasks with a small supervised dataset. With ELMo, we can also achieve similar performance with much less labeled data.

Summary: The language model pre-training is unsupervised and theoretically the pre-training can be scaled up as much as possible since the unlabeled text corpora are abundant. However, it still has the dependency on task-customized models and thus the improvement is only incremental, while searching for a good model architecture for every task remains non-trivial.

4 Cross-View Training#

In ELMo the unsupervised pre-training and task-specific learning happen for two independent models in two separate training stages. **Cross-View Training** (abbr. **CVT**; Clark et al., 2018) combines them into one unified semi-supervised learning procedure

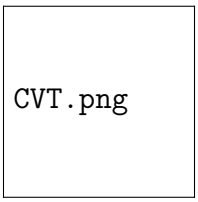


Figure 5: The overview of semi-supervised language model cross-view training. (Image source: original paper)

where the representation of a biLSTM encoder is improved by both supervised learning with labeled data and unsupervised learning with unlabeled data on auxiliary tasks.

4.1 Model Architecture#

The model consists of a two-layer bidirectional LSTM encoder and a primary prediction module. During training, the model is fed with labeled and unlabeled data batches alternatively.

- On *labeled examples*, all the model parameters are updated by standard supervised learning. The loss is the standard cross entropy.
- On *unlabeled examples*, the primary prediction module still can produce a "soft" target, even though we cannot know exactly how accurate they are. In a couple of auxiliary tasks, the predictor only sees and processes a restricted view of the input, such as only using encoder hidden state representation in one direction. The auxiliary task outputs are expected to match the primary prediction target for a full view of input.
  In this way, the encoder is forced to distill the knowledge of the full context into partial representation. At this stage, the biLSTM encoder is backpropagated but the primary prediction module is *fixed*. The loss is to minimize the distance between auxiliary and primary predictions.

4.2 Multi-Task Learning#

When training for multiple tasks simultaneously, CVT adds several extra primary prediction models for additional tasks. They all share the same sentence representation encoder. During supervised training, once one task is randomly selected, parameters in its corresponding predictor and the representation encoder are updated. With unlabeled data samples, the encoder is optimized jointly across all the tasks by minimizing the differences between auxiliary outputs and primary prediction for every task.

The multi-task learning encourages better generality of representation and in the meantime produces a nice side-product: all-tasks-labeled examples from unlabeled data. They are precious data labels considering that cross-task labels are useful but fairly rare.

4.3 Use CVT in Downstream Tasks#

Theoretically the primary prediction module can take any form, generic or task-specific design. The examples presented in the CVT paper include both cases.

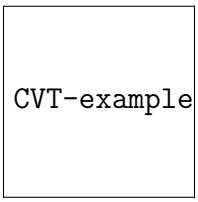


Figure 6: The sequential tagging task depends on four auxiliary prediction models, their inputs only involving hidden states in one direction: forward, backward, future and past. (Image source: original paper)

In sequential tagging tasks (classification for every token) like NER or POS tagging, the predictor module contains two fully connected layers and a softmax layer on the output to produce a probability distribution over class labels. For each token $\mathbf{x}_i$, we take the corresponding hidden states in two layers, $\mathbf{h}_1^{(i)}$ and $\mathbf{h}_2^{(i)}$:

$$ \begin{aligned} p_\theta(y_i \mid \mathbf{x}_i) &= \text{NN}(\mathbf{h}^{(i)}) \\ &= \text{NN}([\mathbf{h}_1^{(i)}; \mathbf{h}_2^{(i)}]) \\ &= \text{softmax}\big(\mathbf{W}\cdot\text{ReLU}(\mathbf{W'}\cdot[\mathbf{h}_1^{(i)}; \mathbf{h}_2^{(i)}]) + \mathbf{b} \big) \end{aligned} $$

The auxiliary tasks are only fed with forward or backward LSTM state in the first layer. Because they only observe partial context, either on the left or right, they have to learn like a language model, trying to predict the next token given the context. The `fwd` and `bwd` auxiliary tasks only take one direction. The `future` and `past` tasks take one step further in forward and backward direction, respectively.

$$ \begin{aligned} p_\theta^\text{fwd}(y_i \mid \mathbf{x}_i) &= \text{NN}^\text{fwd}(\overr \\ p_\theta^\text{bwd}(y_i \mid \mathbf{x}_i) &= \text{NN}^\text{bwd}(\overleftarrow{\mathbf{ \\ p_\theta^\text{future}(y_i \mid \mathbf{x}_i) &= \text{NN}^\text{future}(\overrightarrow{\mat 1)}) \\ p_\theta^\text{past}(y_i \mid \mathbf{x}_i) &= \text{NN}^\text{past}(\overleftarrow{\mat \end{aligned} $$

Note that if the primary prediction module has dropout, the dropout layer works as usual when training with labeled data, but it is not applied when generating "soft" target for auxiliary tasks during training with unlabeled data.

In the machine translation task, the primary prediction module is replaced with a standard unidirectional LSTM decoder with attention. There are two auxiliary tasks: (1) apply dropout on the attention weight vector by randomly zeroing out some values; (2) predict the future word in the target sequence. The primary prediction for auxiliary tasks to match is the best predicted target sequence produced by running the fixed primary decoder on the input sequence with beam search.

5 ULMFiT#

The idea of using generative pretrained LM + task-specific fine-tuning was first explored in ULMFiT (Howard & Ruder, 2018), directly motivated by the success of using ImageNet pre-training for computer vision tasks. The base model is AWD-LSTM.

ULMFiT follows three steps to achieve good transfer learning results on downstream language classification tasks:

1. *General LM pre-training*: on Wikipedia text.

Figure 7: Three training stages of ULMFiT. (Image source: original paper)

2. *Target task LM fine-tuning*: ULMFiT proposed two training techniques for stabilizing the fine-tuning process. See below.

- **Discriminative fine-tuning** is motivated by the fact that different layers of LM capture different types of information (see discussion above). ULMFiT proposed to tune each layer with different learning rates, $\{\eta^1, \dots, \eta^\ell, \dots, \eta^L\}$, where η is the base learning rate for the first layer, η^ℓ is for the ℓ-th layer and there are L layers in total.

- **Slanted triangular learning rates (STLR)** refer to a special learning rate scheduling that first linearly increases the learning rate and then linearly decays it. The increase stage is short so that the model can converge to a parameter space suitable for the task fast, while the decay period is long allowing for better fine-tuning.

3. *Target task classifier fine-tuning*: The pretrained LM is augmented with two standard feed-forward layers and a softmax normalization at the end to predict a target label distribution.

- **Concat pooling** extracts max-polling and mean-pooling over the history of hidden states and concatenates them with the final hidden state.

- **Gradual unfreezing** helps to avoid catastrophic forgetting by gradually unfreezing the model layers starting from the last one. First the last layer is unfrozen and fine-tuned for one epoch. Then the next lower layer is unfrozen. This process is repeated until all the layers are tuned.

6 GPT#

Following the similar idea of ELMo, OpenAI **GPT**, short for **Generative Pre-training Transformer** (Radford et al., 2018), expands the unsupervised language model to a much larger scale by training on a giant collection of free text corpora. Despite of the similarity, GPT has two major differences from ELMo.

1. The model architectures are different: ELMo uses a shallow concatenation of independently trained left-to-right and right-to-left multi-layer LSTMs, while GPT is a multi-layer transformer decoder.

2. The use of contextualized embeddings in downstream tasks are different: ELMo feeds embeddings into models customized for specific tasks as additional features, while GPT fine-tunes the same base model for all end tasks.

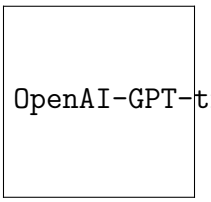


Figure 8: The transformer decoder model architecture in OpenAI GPT.

6.1 Transformer Decoder as Language Model#

Compared to the original transformer architecture, the transformer decoder model discards the encoder part, so there is only one single input sentence rather than two separate source and target sequences.

This model applies multiple transformer blocks over the embeddings of input sequences. Each block contains a masked *multi-headed self-attention* layer and a *pointwise feed-forward* layer. The final output produces a distribution over target tokens after softmax normalization.

The loss is the negative log-likelihood, same as ELMo, but without backward computation. Let's say, the context window of the size k is located before the target word and the loss would look like:

$$ \mathcal{L}_\text{LM} = -\sum_{i} \log p(x_i\mid x_{i-k}, \dots, x_{i-1}) $$

6.2 Byte Pair Encoding#

Byte Pair Encoding (**BPE**) is used to encode the input sequences. BPE was originally proposed as a data compression algorithm in 1990s and then was adopted to solve the open-vocabulary issue in machine translation, as we can easily run into rare and unknown words when translating into a new language. Motivated by the intuition that rare and unknown words can often be decomposed into multiple subwords, BPE finds the best word segmentation by iteratively and greedily merging frequent pairs of characters.

6.3 Supervised Fine-Tuning#

The most substantial upgrade that OpenAI GPT proposed is to get rid of the task-specific model and use the pre-trained language model directly!

Let's take classification as an example. Say, in the labeled dataset, each input has n tokens, $\mathbf{x} = (x_1, \dots, x_n)$, and one label y. GPT first processes the input sequence $\mathbf{x}$ through the pre-trained transformer decoder and the last layer output for the last token x_n is $\mathbf{h}_L^{(n)}$. Then with only one new trainable weight matrix $\mathbf{W}_y$, it can predict a distribution over class labels.

GPT-classification.png

e-3e-3 e-3e-3$$ e-3e-3The e-3e-3 $$ P(y\mid x_1, \dots, x_n) = \text{softmax}(\mathbf{h}_L^{(n)}\mathbf{W}_y) $$ loss is to minimize the negative log-likelihood for true labels. In addition, adding the LM loss as an auxiliary loss is found to be beneficial, because:

- (1) it helps accelerate convergence during training and
- (2) it is expected to improve the generalization of the supervised model.

e-3e-3$$ e-3e-3 $$ \begin{aligned} \mathcal{L}_1 &= \sum_{(\mathbf{x}, y) \in \mathcal{D}} \log P(y\mid x_1, \dots, x_n) = \sum_{(\mathbf{x}, y) \in \mathcal{D}} \log \text{softmax}(\mathbf{h}_L^{(n)}\mathbf{W}_y) \\ \mathcal{L}_2 &= -\sum_{i} \log p(x_i\mid x_{i-k}, \dots, x_{i-1}) \\ \mathcal{L} &= \mathcal{L}_1 + \lambda \mathcal{L}_2 \end{aligned} $$ With similar signs, no customized model structure is needed for other tasks (see Fig. 7). If the task input contains multiple sentences, a special delimiter token ($) is added between each pair of

GPT-downstream-tasks.png

e-3e-3For [t]5.0e-3 Training objects in slightly modified GPT transformer the sentence similarity task, because the ordering does not matter, both orderings are included. For the multiple choice task, the context is paired with every answer candidate.

models for downstream tasks. (Image source: original paper)

e-3e-3**Summary**e-3e-3

It is super neat and encouraging to see that such a general framework is capable to beat SOTA on most language tasks at that time (June 2018). At the first stage, generative pre-training of a language model can

e-3e-3One limitation of GPT is its uni-directional nature — the model is only trained to predict the future left-to-right context.e-3e-3

7 BERT

transformer-encoder-2.png

Recap of Transformer

BERT, short for **Bidirectional Encoder Representations from Transformers** (Devlin, et al., 2019) is a direct descendant to GPT: train a large language model on free text and then fine-tune on specific tasks with-

Compared to GPT, the largest difference and improvement of BERT is to make training **bi-directional**. The model learns to predict both context on the left and right. The paper according to the ablation study claimed that:

“bidirectional nature of our model is the single most important new contribution”

7.1 Pre-training Tasks#

The model architecture of BERT is a multi-layer bidirectional Transformer encoder.

Encoder model architecture. (Image source: Transformer paper) e-3e-3Toe-3e-3*Task 3k-3 encourage the bidirectional prediction and sentence-level understanding, BERT is trained with two tasks instead of the basic language task (that is, to predict the next token given context).

Task 1: Mask language model (MLM)

From Wikiped "A cloze test (also cloze deletion test) is an exercise, test, or assessment consisting of a portion of language with certain items, words, or signs removed (cloze text), where the participant is asked to replace

language-model-comparison.png

Comparison

It is unsurprising to believe that a representation that learns the context around a word rather than just after the word is able to better capture its meaning, both syntactically and seman-

Task 2: Next sentence prediction

Randomly mask 15% of tokens in each sequence. Because if we only replace masked tokens with a special placeholder `[MASK]`, the special token would never be encountered during fine-tuning. Hence, BERT employed sev-

Motivated by the fact that many downstream tasks involve the understanding of relationships between sentences (i.e., QA, NLI), BERT added another auxiliary task on training a *binary classifier* for telling whether one

The training data for both auxiliary tasks above can be trivially generated from any monolingual corpus. Hence the scale of training is unbounded.

The model processes both sentences and output a binary

Sample sentence pairs (A, B) so that:

- (a) 50% of the time, B follows A;
- (b) 50% of the time, B does not follow A.

The training loss is the sum of the mean masked LM likelihood

of BERT, OpenAI GPT and ELMo model architectures. (Image source: original paper)

e-3e-3 e-3e-3Thœ-3e-3 1.[t]5.0e-3 BERT-input-embedding.png BERT input representation. (Image

7.2 Input Embedding#

Input embedding is the sum of three parts:

WordPiece tokenization embeddings: The WordPiece model was originally proposed for Japanese or Korean segmentation problem. Instead of using naturally split English word, they can be further di-

source: original paper)

Note that the first token is always forced to be `[CLS]` — a placeholder that will be used later for prediction in downstream tasks.

7.3 Use BERT in Downstream Tasks#

Fine-tuning requires only a few new parameters added, just like OpenAI GPT.

For classification tasks, we get the prediction by taking the final hidden state of the special first token `[CLS]`, $\mathbf{h}^\text{[CLS]}_L$, and multiplying it with a small weight matrix, $\text{softmax}(\mathbf{h}^\text{[CLS]}_L \mathbf{W}_\text{cls})$.

For QA tasks like SQuAD, we need to predict the text span in the given paragraph for an given question. BERT predicts two probability distributions of every token, being the start and the end of the text span. Only two new parameters are learned during fine-tuning: $\mathbf{W}_\text{start}$ and $\mathbf{W}_\text{end}$. The probability of word i being the start of the answer span is computed as a dot product between its final hidden vector and the start vector, followed by a softmax over all words: $P_i = \frac{e^{\mathbf{h}^\text{[CLS]}_L \mathbf{W}_\text{start}}}{\sum_j e^{\mathbf{h}^\text{[CLS]}_L \mathbf{W}_\text{start}}}$. The same formula is used for the end of the text span. The score of a candidate span from position i to position j is computed as $\mathbf{h}^\text{[CLS]}_L \mathbf{W}_\text{start} + \mathbf{h}^\text{[CLS]}_L \mathbf{W}_\text{end}$, and the maximum scoring span where $j \geq i$ is used as a prediction.

Overall the add-on part for end task fine-tuning is very minimal — one or two weight matrices to convert the Transformer hidden states to an interpretable format. Check the paper for implementation details for two

Training objects in slightly modified BERT models for downstream tasks.

(Image source: original paper)

summary table compares differences between fine-tuning of OpenAI GPT and BERT.	**OpenAI GPT** | **BERT** | | Special char | `[SEP]` and `[CLS]` are only introduced at fine-tuning stage. | `[SEP]` and `[CLS]` and sentence A/B embeddings are learned at the pre-training stage. | | Training process | 1M steps, batch	**8 ALBERT** # (Lan et al. 2019), short for **A Lite BERT**, is a light-weighted version of BERT model. An ALBERT model can be trained 1.7x faster with 18x fewer parameters, compared to a BERT model of similar configuration. ALBERT in-	**ALBERT** # **Factorized Embedding Parameterization** # In BERT, the WordPiece tokenization embedding size E is configured to be the same as the hidden state size H. That is saying, if we want to increase the model size (larger H), we need to learn a larger tokenization embedding whose size	Conceptually, because the tokenization embedding is expected to learn *context-independent* representation and the hidden states are *context-dependent*, it makes sense to separate the size of the hidden layers from the size of vocabu-

e-3e-3 e-3e-3Parameter e-3e-3Inter-sentence e-3e-3Negative e-3e-3For e-3e-3The e-3e-3The e-3e-3

8.2 Cross-layer Parameter Sharing#

Parameters can be shared in many ways: (a) only share feed-forward part; (b) only share attention parameters; or (c) share all the parameters. This technique reduces the number of parameters by a ton

8.3 Sentence-Order Prediction (SOP)#

Inter-sentence prediction (NSP) task of BERT turned out to be too easy. ALBERT instead adopted a sentence-order prediction (SOP) self-supervised loss,

- Positive sample: two consecutive segments from the same document.
- Negative sample: same as above, but the segment order is switched.

9 GPT-2#

9.1 Zero-Shot Transfer#

OpenAI GPT-2 language model is a direct successor to GPT. GPT-2 has 1.5B parameters, 10x more than the original GPT, and it achieves SOTA results on 7 out of 8 tested language modeling datasets in a

- Text generation is straightforward using LM.
- Machine translation task, for example, English to Chinese, is induced by conditioning LM on pairs of "English sentence = Chinese sentence" and "the tar-

9.2 BPE on Byte Sequences#

Same as the original BPE, but GPT-2 uses BPE on UTF-8 byte sequences. Each byte can represent 256 different values in 8 bits, while UTF-8 can use up to 4 bytes for one character, supporting up to 2^{31} characters. Using the byte sequence representation, GPT-2 is able to assign a probability to any Unicode string, regardless of any preprocessing steps.

BPE merges frequently co-occurred byte pairs in a greedy manner. To prevent it from generating multiple versions of common words (i.e. `dog.`, `dog!` and `dog?` for the word `dog`), GPT-2 prevents BPE from merging characters across categories.

9.3 Model Modifications#

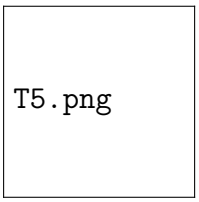
Compared to GPT, other than having more transformer layers and parameters, GPT-2 incorporates only a few architecture modifications:

- Layer normalization was moved to the input of each sub-block, similar to a residual unit of type "building block" (differently from the original type "bottleneck", it has batch normalization applied before weight

10 RoBERTa#

RoBERTa (short for **R**obustly **o**ptimized **BERT** **a**pproach; Liu, et al. 2019) refers to a new receipt for training BERT to achieve better results, as they found that the original BERT model is significantly undertrained. The receipt contains the following:

1. Train for longer with bigger batch size.
2. Remove the next sentence prediction (NSP) task.
3. Use longer sequences in training data format.
4. Dynamically change the masking pattern applied to the training data. The original BERT was pretrained with masking and static patterns, so that the same masked positions in sequences were fed to the model in different epochs. They also found that using dynamic masking is helpful when training for longer and with bigger batch size.
5. Use a new dataset CommonCrawl News and further confirmed that pretraining with *more data helps* improve performance on downstream tasks. It was pretrained with BPE on byte-level subwords, similar to GPT-2. They also found that choices

e-3e-3 e-3e-3The [t]5.0e-3 

A diagram of T5 task evaluation. The

11 T5#

language model **T5** is short for **"Text-to-Text Transfer Transformer"** (Raffel et al., 2020). The encoder-decoder implementation follows the original Transformer architecture: tokens → embedding → encoder

text-to-text framework casts every task into a generic form: feeding input text to predict

some target text. (Image source: Raffel et al., 2020)

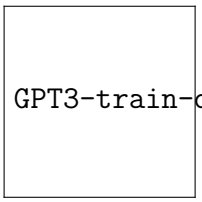
e-3e-3The model is trained on Web corpus extracted from Apr 2019 with various filters applied. The model is fine-tuned for each downstream task separately via "adapter layers" (add an extra layer for training) or "gradual unfreezing" (see

e-3e-3As the authors mentioned in the paper "...our goal is not to propose new methods but instead to provide a comprehensive perspective on where the field stands", the T5 long paper described a lot of training setup and

e-3e-3

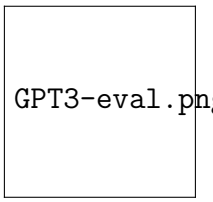
12 GPT 3#

e-3e-3**GPT-3** (Brown et al., 2020) has the same architecture as GPT-2 but contains 175B parameters, 10x larger than GPT-2 (1.5B). In addition, GPT-3 uses alternating dense and locally banded sparse atten-

[t]5.0e-3 

Training datasets for

GPT-3. Note that the occurrence of each dataset during training is not proportional to



the dataset size. (Table source: Brown et al., 2020)

e-3e-3For [t]5.0e-3 all the downstream evaluation, GPT-3 is tested in the few-shot setting without any gradient-based fine-tuning. Here the few-shot examples are provided as part of the prompt. GPT-3 achieves strong performance on

The evaluation performance increases with the model size and the number of examples.

(Image source: Brown et al., 2020) e-3e-3 e-3e-3The e-3XLNet$$e-3where e-3Note e-3However,

13 XLNet

Autoregressive (AR) model such as GPT and autoencoder (AE) model such as BERT are two most common ways for language modeling. However, each has their own disadvantages: AR does not learn the bidirectional context, which is

(Yang et al. 2019) generalizes the AE method to incorporate the benefits of AR. XLNet proposed the **permutation language modeling** objective. For a text sequence, it samples a factorization

$$\begin{aligned} \mathcal{L}_\text{XLNet} &= - \mathbb{E}_{\mathbf{z} \sim \mathcal{Z}_T} \Big[\sum_{t=1}^T \log p_\theta (X_{z_t} = x \mid \mathbf{x}_{\mathbf{z}_{<{t}}}) \Big] \\ &= - \mathbb{E}_{\mathbf{z} \sim \mathcal{Z}_T} \Big[\log \frac{ \exp(e(x)^\top \color{red}{h_\theta (\mathbf{x}_{\mathbf{z}_{<{t}}})}) }{ \sum_{x'} \exp(e(x')^\top \color{red}{h_\theta (\mathbf{x}_{\mathbf{z}_{<{t}}})}) } \Big] \\ &= - \mathbb{E}_{\mathbf{z} \sim \mathcal{Z}_T} \Big[\log \frac{ \exp(e(x)^\top \color{blue}{g_\theta (\mathbf{x}_{\mathbf{z}_{<{t}}}, z_t)}) }{ \sum_{x'} \exp(e(x')^\top \color{blue}{g_\theta \end{aligned}$$

where $\mathcal{Z}_T$ is a set of all possible permutation of length T; z_t and $\mathbf{z}_{<t}$ denote the t-th element and the first $t-1$ elements of a permutation $\mathbf{z} \in \mathcal{Z}_T$.

Note that a naive representation of the hidden state of the context, $h_\theta (\mathbf{x}_{\mathbf{z}_{<t}})$ in red, does not depend on which position the model tries to predict, as the permutation $\mathbf{z}$ breaks the default ordering. Therefore, XLNet

However, two different requirements on $g_\theta (\mathbf{x}_{\mathbf{z}_{<t}}, z_t)$ lead to a two-stream self-attention design to accommodate:

e-3e-3 [t]5.0e-3 XLNet-two-stream-attention.png The illustration of two-stream self-attention mechanism 1. When predicting x_{z_t}, it should only encode the position z_t but not the content x_{z_t}; otherwise it is trivial. This is wrapped into the "query representation" $g_{z_t} = g_\theta (\mathbf{x}_{\mathbf{z}_{<t}}, z_t)$ does not en-

in XLNet. (Image source: Yang et al. 2019)

e-3e-3Conceptually, the two streams of representations are updated as follows,

e-3$$
\begin{aligned}
g_{z_t}^{(m)} &\gets \text{Attention}(Q = g^{(m-1)}_{z_t}, KV=\mathbf{h}^{(m-1)}_{\color{red}{\mathbf{z}_{<t}}}; \theta) &\text{(query stream: use }z_t\text{ but cannot see }x_{z_t}\text{)}\\
h_{z_t}^{(m)} &\gets \text{Attention}(Q = h^{(m-1)}_{z_t}, KV=\mathbf{h}^{(m-1)}_{\color{blue}{\mathbf{z}_{\leq t}}}; \theta) &\text{(content stream: use both }z_t\text{ and }x_{z_t}\text{)}\\
\end{aligned}
$$

e-3e-3Given the difficulty of optimization in permutation language modeling, XLNet is set to only predict the last chunk of tokens in a factorization order.

e-3e-3The [t]5.0e-3 

Comparison of model performance of XLNet with a couple other language models on GLUE, all single-task, no ensembles. (Image source:

name in XL-Net ac-tu-ally comes from Transformer-XL. It in-cor-po-rates the de-sign of Transformer-XL to ex-tend the at-ten-tion span by reusing hid-den states from pre-vi-ous seg-ments.

Yang et al. 2019) e-3e-3 e-3e-3**BART** [t]5.0e-3

14 BART#

(Lewis et al., 2019) is a denoising autoencoder to recover the original text from a randomly corrupted version. It combines **B**idirectional and **A**uto**R**egressive **T**ransformer: precisely, jointly training BERT-like bidirectional encoder and GPT-like autore-

A schematic comparison of BART with BERT and GPT. (Image source:

Lewis et al., 2019)	e-3	e-3	The[t]	5.0e-3	 Comparison of different language ex- per- i- mented with a va- ri- ety of nois- ing trans- for- ma- tions, in- clud- ing to- ken mask- ing, to- ken dele- tion, text in- fill- ing (i.e. A ran- domly sam- pled text span, which may con- tain mul- ti- ple to- kens,

modeling pre-training objectives. (Image source: Lewis et al., 2019)

e-3e-3Learnings3e-3

from their experiments:

- The performance of pre-training methods varies significantly across downstream tasks.
- Token masking is crucial, as the performance is poor when only sentence permutation or document rotation is ap-

ELECTRA-overview.png

e-3e-3 e-3e-3Most[t]5.0e-3 Illustration of ELECTRA model architecture.

15 ELECTRA#

current pretraining large language models demand a lot of computation resources, raising concerns about their cost and accessibility. **ELECTRA** ("Efficiently Learning an Encoder that Classifies To-

(Image source: Clark et al. 2020)

ELECTRA proposes a new pretraining task, called "Replaced Token Detection" (RTD). Let's randomly sample k positions to be masked. Each selected token in the original text is replaced by a plausible alternative

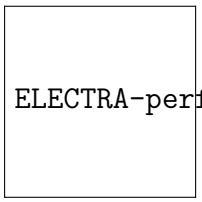
$$
\begin{aligned}
\boldsymbol{m} &= [m_1, \dots, m_k] \text{ where } m_i \sim \text{unif}\{1,n\}\text{ for } i=1, \dots, k \\
\boldsymbol{x}^\text{masked} &= \text{REPLACE}(\boldsymbol{x}, \boldsymbol{m}, \texttt{[MASK]}) \\
\boldsymbol{x}^\text{corrupt} &= \text{REPLACE}(\boldsymbol{x}, \boldsymbol{m}, \tilde{\boldsymbol{x}}) \text{ where } \tilde{x}_i \sim p_G(x_i \mid \boldsymbol{x}^\text{masked}) \text{ for } i \in \boldsymbol{m} \\
\end{aligned}
$$

The loss for the generator is the negative log-likelihood just as in other language models. The loss for the discriminator is the cross-entropy. Note that the generator is not adversarially trained to fool

$$
\begin{aligned}
\mathcal{L}_\text{MLM}(\mathbf{x}, \theta_G) &= \mathbb{E}\Big(\sum_{i \in \boldsymbol{m}} -\log p_G (x_i \mid \boldsymbol{x}^\text{masked}) \Big) \\
\mathcal{L}_\text{Disc}(\mathbf{x}, \theta_D) &= \mathbb{E}\Big(\mathbb{1}[x^\text{corrupt}_t = x_t] \log D(\boldsymbol{x}^\text{corrupt}, t) + \mathbb{1}[x^\text{corrupt}_t \neq x_t] \log (1 - \log D(\boldsymbol{x}^\text{corrupt}, t)) \Big)
\end{aligned}
$$

They found it more beneficial to only share the embeddings between generator and discriminator while using small generator (1/4 to 1/2 the discriminator size), rather than sharing all the weights

After pretraining the generator is discarded and only the ELECTRA discriminator is fine-tuned further for downstream tasks. The following table shows ELECTRA's performance on the GLUE dev set.



Comparison of ELECTRA with other language models on the GLUE dev

set. (Image source: Clark et al. 2020)

16 Summary

Pre-training Tasks	# of model used as an intrinsic evaluation metric for gauging how well a language model can capture the real word distribution conditioned on the context.
CoVe seq2seq NMT model supervised learning using translation dataset.	
ELMo two-layer biLSTM next token prediction	
CVT two-layer biLSTM semi-supervised learning using both labeled and unlabeled datasets	
ULMFiT	

17 Metric: Perplexity

Perplexity of a discrete proability distribution p is defined as the exponentiation of the entropy:

$$2^{H(p)} = 2^{-\sum_x p(x) \log_2 p(x)}$$

Given a sentence with N words, $s = (w_1, \dots, w_N)$, the entropy looks as follows, simply assuming that each word has the same frequency, $\frac{1}{N}$:

$$
\begin{aligned}
H(s) &= -\sum_{i=1}^N P(w_i) \log_2 p(w_i) \\
&= -\sum_{i=1}^N \frac{1}{N} \log_2 p(w_i)
\end{aligned}
$$

The perplexity for the sentence becomes:

$$
\begin{aligned}
2^{H(s)} &= 2^{-\frac{1}{N}\sum_{i=1}^N \log_2 p(w_i)} \\
&= (2^{\sum_{i=1}^N \log_2 p(w_i)})^{\frac{1}{N}} \\
&= (p(w_1) \dots p(w_N))^{\frac{1}{N}}
\end{aligned}
$$

A good language model should predict high probabilities for real sentences, so the smaller the perplexity the better.

18 Common Tasks and Datasets

Question-Answering

- SQuAD (Stanford Question Answering Dataset): A reading comprehension dataset, consisting of questions posed on a set of Wikipedia articles, where the answer to every question is a span of text.
- RACE (ReAding Com-

Commonsense

Natural Language Inference (NLI): also known as "Text Entailment", an exercise to discern in logic whether one sentence can be inferred from another.

- Story Cloze Test: A commonsense reasoning framework for evaluating story understanding and generation. The test requires a system to choose the correct ending to multi-sentence stories from two options.
-

- **Named Entity Recognition (NER)**: A task: consists of newswire from the Reuters, concentrating on four types of named entities: persons, locations, organizations and names of miscellaneous entities.
- **Semantic Role Labeling (SRL)**: models the predicate-argument structure of a sentence, and it often described as answering “Who did what to whom”.
- **Semantic similarity** (also known as *paraphrase detection*): It annotate pairs of sentences for semantic equivalence. Data is extracted from news sources on the web, with annotations indicating whether each pair is semantically equivalent.
- **Linguistic Acceptability**: a task to annotate if a sentence is grammatically acceptable.

RTE (Recognizing Textual Entailment): A set of datasets initiated by text entailment challenges.

SNLI (Stanford Natural Language Inference): A collection of 570k human-written English sentence pairs manually

CoNLL 2003 NER: consists of newswire from the Reuters, concentrating on four types of named entities: persons, locations, organizations and names of miscellaneous entities.

OntoNotes 5.0: This corpus contains

SST (Stanford Sentiment Treebank): IMDB: A large dataset of movie reviews with binary sentiment classification labels.

CoNLL-2004 & CoNLL-2005:

MRPC (Microsoft Research Paraphrase Corpus): It is extracted from news sources on the web, with annotations indicating whether each pair is semantically equivalent.

QQP (Quora Question Pairs) STS Benchmark:

CoLA (Corpus of Linguistic Acceptability): a binary single-sentence classification task.

- **Text Chunking**: To divide a text in syntactically correlated parts of words.
- **Part-of-Speech (POS) Tagging**: tag parts of speech to each token, such as noun, verb, adjective, etc. the Wall Street Journal portion of the Penn Treebank (Marcus et al., 1993).
- **Machine Translation**: See Standard NLP page.
- **Coreference Resolution**: To cluster mentions in text that refer to the same underlying real world entities.
  - WMT 2015 English-Czech data (Large)
  - WMT 2014 English-German data (Medium)
  - IWSLT 2015 English-Vietnamese data (Small)
- **Long-range Dependency**
  - LAMBADA (Language Modeling Broadened to Account for Discourse Aspects): A collection of narrative passages extracted from the BookCorpus and the task is to predict the last word, which require at least 50 tokens of
- **Multi-task Benchmark**
  - GLUE multi-task benchmark: https://g
  - decaNLP benchmark: https://d

Unsupervised pre-training dataset: Books corpus: The corpus contains "over 7,000 unique unpublished books from a variety of genres including Adventure, Fantasy, and Romance."

- 1B Word Language Model Benchmark
- English Wikipedia: ~2500M words

Cited as:

```
@article{weng2019LM,
  title   = "Generalized Language Models",
  author  = "Weng, Lilian",
  journal = "Lilianweng.github.io",
  year    = "2019",
  url     = "https://lilianweng.github.io/posts/2019-0
}
```

19 References

[1] Yoshua Bengio, et al. "A neural probabilistic language model." Journal of machine learning research 3.Feb (2003): 1137-1155.

[2] Ronan Collobert, et al. "Natural language processing (almost) from scratch." Journal of Machine Learning Research 12.Aug (2011): 2493-2537.

[3] Andrew M. Dai, et al. "Semi-supervised sequence learning." NIPS. 2015.

[4] Matthew E. Peters, et al. "Deep contextualized word representations." NAACL-HLT 2017.

[5] OpenAI Blog "Improving Language Understanding with Unsupervised Learning", June 11, 2018.

[6] OpenAI Blog "Better Language Models and Their Implications." Feb 14, 2019.

e-3e-3[6]e-3e-3[7]e-3e-3[8]e-3e-3[9]e-3e-3[10]e-3e-3[11]e-3e-3[12]e-3e-3[13]e-3e-3[14]e-3e-3[15]e-3e-3[16]

Jeremy Howard and Sebastian Ruder. "Universal language model fine-tuning for text classification." ACL 2018.

Alec Radford et al. "Improving Language Understanding by Generative Pre-Training" OpenAI Blog, June 11, 2018.

Jacob Devlin, et al. "BERT: Pre-training of deep bidirectional transformers for language understanding." arXiv:1810.04805 (2018).

Mike Schuster, and Kaisuke Nakajima. "Japanese and Korean voice search." ICASSP 2012.

Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation

Ashish Vaswani, et al. "Attention is all you need." NIPS 2017.

Peter J. Liu, et al. "Generating wikipedia by summarizing long sequences." ICLR 2018.

Sebastian Ruder. "10 Exciting Ideas of 2018 in NLP" Dec 2018.

Alec Radford, et al. "Language Models are Unsupervised Multitask Learners.". 2019.

Rico Sennrich, et al. "Neural machine translation of rare words with subword units." arXiv preprint arXiv:1508.07909. 2015.

Zhenzhong Lan, et al. "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations." arXiv Preprint arXiv:1909.119 (2019).

[17] Yinhan Liu, et al. "RoBERTa: A Robustly Optimized BERT Pretraining Approach." arXiv Preprint arXiv:1907.11692 (2019).

[18] Tom B Brown, et al. "Language Models are Few-Shot Learners" NeuriPS 2020.

[19] Zhilin Yang et al. "XLNet: Generalized Autoregressive Pretraining for Language Understanding." NeuriPS 2019.

[20] Mike Lewis et al. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension." ACL 2020.

[21] Kevin Clark et al. "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators." ICLR 2020.

[22] Colin Raffel, et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" JMLR 2020.

- Architecture
- Nlp
- Long-Read
- Transformer
- Attention
- Language-Model

- Attention? Attention!
- Deep Neural Networks Dramatically Overfitted
- Object Detection Part 4: Fast Detection Models

20 Citation

Please cite this work as:

> Weng, Lilian. "Title". Lil 'Log (May 2025). https://lilianweng.github.io/posts/2019-01-31-lm/

Or use the BibTeX citation:

```
@article{weng2025,
title = {},
author = {Weng, Lilian},
journal = {lilianweng.github.io},
year = {2025}, month = {May},
url = {}
}
```

21 References

Tags: tag1 tag2

⟨ **Title** **Title** ⟩

Share this article: Twitter | LinkedIn | Reddit | Facebook | WhatsApp | Telegram